This will create `logout_path` and `logout_url` as named helpers in your application.
Calling `logout_path` will return `/exit`
You can also use this to override routing methods defined by resources, like this:

```
get ':username', to: 'users#show', as: :user
```

This will define a `user_path` method that will be available in controllers, helpers and views that will go to a route such as `/bob`. Inside the `show` action of `UsersController`, `params[:username]` will contain the username for the user. Change `:username` in the route definition if you do not want your parameter name to be `:username`.

3.7 HTTP Verb Constraints

In general, you should use the `get`, `post`, `put`, `patch` and `delete` methods to constrain a route to a particular verb. You can use the `match` method with the `:via` option to match multiple verbs at once:

```
match 'photos', to: 'photos#show', via: [:get, :post]
```

You can match all verbs to a particular route using `via: :all`:

```
match 'photos', to: 'photos#show', via: :all
```

Routing both `GET` and `POST` requests to a single action has security implications. In general, you should avoid routing all verbs to an action unless you have a good reason to.

'GET' in Rails won't check for CSRF token. You should never write to the database from 'GET' requests, for more information see the security guide on CSRF countermeasures.

3.8 Segment Constraints

You can use the `:constraints` option to enforce a format for a dynamic segment:

```
get 'photos/:id', to: 'photos#show', constraints: { id: /[A-Z]\d{5}/ }
```

This route would match paths such as `/photos/A12345`, but not `/photos/893`. You can more succinctly express the same route this way:

```
get 'photos/:id', to: 'photos#show', id: /[A-Z]\d{5}/
```

`:constraints` takes regular expressions with the restriction that regexp anchors can't be used. For example, the following route will not work:

```
get '/:id', to: 'articles#show', constraints: { id: /^\d/ }
```

However, note that you don't need to use anchors because all routes are anchored at the start.

For example, the following routes would allow for `articles` with `to_param` values like `1-hello-world` that always begin with a number and `users` with `to_param` values like `david` that never begin with a number to share the root namespace:

```
get '/:id', to: 'articles#show', constraints: { id: /\d.+/ }
get '/:username', to: 'users#show'
```

3.9 Request-Based Constraints

You can also constrain a route based on any method on the Request object that returns a `String`.
You specify a request-based constraint the same way that you specify a segment constraint:

```
get 'photos', to: 'photos#index', constraints: { subdomain: 'admin' }
```

You can also specify constraints in a block form:

```
namespace :admin do
  constraints subdomain: 'admin' do
    resources :photos
```